

.NET on Debian で 同人活動に優しい SNSの開発 基礎編

サークル
Meguri

目次

はじめに.....	4
第 1 章 AWS EC2 Debian 12 のセットアップ.....	5
第 2 章 Apache2 のセットアップ.....	7
インストール.....	7
SSL の設定.....	7
リバースプロキシを使ったアプリケーションの公開.....	8
第 3 章 PostgreSQL のセットアップ.....	11
開発環境 Windows.....	11
運用環境 Debian 12.....	12
第 4 章 Postfix / Dovecot のセットアップ.....	13
Postfix インストール.....	13
Dovecot インストール.....	13
メールアカウント登録.....	13
SSL/TLS の設定.....	14
SPF の設定.....	14
DKIM の設定.....	15
DMARC の設定.....	18
第 5 章 Web アプリケーション開発.....	20
.NET 8 SDK のインストール.....	20
プロジェクトの作成.....	20
コードファーストマイグレーションの実行.....	21
認証.....	21
メール送信.....	22
多要素認証の QR コード生成.....	25
認可.....	25
モデルの修正.....	25
要件クラスと認可ハンドラーの作成.....	26
サービスコレクションへの登録.....	28
コントローラーの修正.....	29
画像のアップロードとデータベースへの格納.....	33
モデルの修正.....	33
ビューの修正.....	34
コントローラーの修正.....	35
Meguri でのコンテンツの分類.....	38

はじめに

同人活動に優しい SNS とは、どういうものでしょうか？ この問いに正解があるわけではありませんが、本サークルでは、下記のような原理原則を持つものが良いのではないかと考えています。

1. 規制よりもゾーニング
2. 短文投稿、動画投稿、イラスト投稿などの個別ジャンル対応ではなく、オールジャンル対応
3. フォロワー数やいいねの数を競うことよりも、ユーザーの間の交流が主体の場合

本書は、これらの実現を目指す SNS、Meguri の開発についての解説書になります。本書は基礎編として、主に Debian での環境構築、ASP.NET MVC での認証認可、画像のデータベースへの格納など、SNS で必要とされる基礎的な機能の実装について解説しています。

予定は未定ですが、応用編では、上記の原理原則を実現する具体的な UI や機能の実装について、解説できたらいいなと思っています。

2024 年 8 月 12 日 サークル Meguri

【SNS Meguri (開発中)】

<https://meguri.info/>

【GitHub】

<https://github.com/megurikuru/meguri>

第 1 章 AWS EC2 Debian 12 のセットアップ

運用環境には、AWS EC2 上の Debian インスタンスを使用します。楽なのは Azure ですが、Azure の App Service も、SQL データベースもお安くはないので。

Debian 12 を選択して EC2 インスタンスを作成します。


Debian
無料利用枠の対象
検証済みプロバイダー

Debian 12 (HVM), SSD Volume Type
ami-08c2888d01ed84209 (64 ビット (x86)) / ami-08c4a973a3ea147c4 (64 ビット (Arm))
Debian 12 (HVM), EBS General Purpose (SSD) Volume Type. Community developed free GNU/Linux distribution.
<https://www.debian.org/>
プラットフォーム: debian ルートデバイスタイプ: ebs 仮想化: hvm ENA 有効: はい

選択
☒ 64 ビット (x86)
☐ 64 ビット (Arm)

インスタンスタイプは、t3.small または、t3.micro を選択します。開発時は、t3.micro でもなんとか大丈夫です。下表は t3.small と t3.micro の比較です。

属性	t3.small	t3.micro
オンデマンド Linux 価格設定	0.0272USD1 時間あたり	0.0136USD1 時間あたり
Windows のオンデマンド価格設定	0.0456USD1 時間あたり	0.0228USD1 時間あたり
vCPU	2(1コア)	2(1コア)
メモリ (MiB)	2048	1024

インスタンスにログインしたら、APT コマンドで、最新のパッケージ情報のダウンロードとパッケージのアップグレードを行います。

```
# apt update
# apt -y upgrade
```

日本語関係のパッケージのインストールとロケールの設定などを行います。

```
# apt -y install task-japanese locales-all
# localectl set-locale LANG=ja_JP.UTF-8 LANGUAGE="ja_JP:ja"
# source /etc/default/locale
# echo $LANG
ja_JP.UTF-8
# timedatectl set-timezone Asia/Tokyo
# date
2024年 1月 23日 火曜日 22:09:05 JST
```

1GBのスワップ領域を設定します。特にt3.microの場合はメモリが1GBしかないので必要だと思います。もちろん、運用時にはスワップ領域が使われるようではダメですが。

```
# dd if=/dev/zero of=/swapfile bs=1M count=1024
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 5.06093 s, 212 MB/s
# chmod 600 /swapfile
# mkswap /swapfile
Setting up swspace version 1, size = 1024 MiB (1073737728 bytes)
no label, UUID=4bcd24bf-2c0b-4a50-ad31-e99e33857bc3
# swapon /swapfile
# echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
/swapfile none swap sw 0 0
# free -m
```

	total	used	free	shared	buff/cache	available
Mem:	949	541	254	31	320	407
Swap:	1023	0	1023			

第 2 章 Apache2 のセットアップ

インストール

Web サーバーには Apache2 を使用します。Apache2 のインストールについては下記ページを参考にしてください。

Debian 12 Bookworm : Apache2 : インストール : Server World
https://www.server-world.info/query?os=Debian_12&p=httpd&f=1

SSL の設定

Let's Encrypt を使用して SSL の設定を行います。SSL の設定については下記ページを参考にしてください。

Debian 12 Bookworm : SSL 証明書を取得する (Let's Encrypt) : Server World
https://www.server-world.info/query?os=Debian_12&p=ssl&f=2

本書では一部の手順を記載します。

Certbot クライアントをインストールします。

```
# apt -y install certbot
```

証明書を取得します。meguri.info の部分は、自身が使用するドメインに変更してください。

```
# certbot certonly --standalone -d meguri.info
```

Apache の設定ファイルに SSL の設定を記述します。下記の設定ファイルは、リバースプロキシの設定も含んでいます。リバースプロキシについては後で説明します。

```
# cd /etc/apache2/sites-enabled
# vim meguri.conf
<VirtualHost *:80>
    ServerName meguri.info

    RequestHeader set "X-Forwarded-Proto" expr=%{REQUEST_SCHEME}s
    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    ErrorLog ${APACHE_LOG_DIR}/meguri-error.log
    CustomLog ${APACHE_LOG_DIR}/meguri-access.log common
</VirtualHost>
```

```
<VirtualHost *:443>
    ServerName meguri.info

    SSLEngine on
    SSLProtocol all -SSLv3
    SSLProxyProtocol all -SSLv3
    SSLHonorCipherOrder on
    SSLCertificateFile /etc/letsencrypt/live/meguri.info/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/meguri.info/privkey.pem

    RequestHeader set "X-Forwarded-Proto" expr=%{REQUEST_SCHEME}s
    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    ErrorLog ${APACHE_LOG_DIR}/meguri-ssl-error.log
    CustomLog ${APACHE_LOG_DIR}/meguri-ssl-access.log common
</VirtualHost>
```

Let's Encrypt の証明書の有効期間は 90 日間しかないので自動更新の設定を行います。
証明書の自動更新スクリプトを作成します。

```
# cd /usr/sbin
# vim certbot-renew.sh

systemctl stop apache2
certbot renew
systemctl start apache2
# chmod 755 certbot-renew.sh
```

cron に設定します。

```
# crontab -e

35 4 * * * /usr/sbin/certbot-renew.sh
```

リバースプロキシを使ったアプリケーションの公開

Apache をリバースプロキシとしてアプリケーションを公開します。リバースプロキシとは、プロキシサーバーとは反対に外部ネットワークから内部ネットワークへの接続を中継するサーバーです。ASP.NET Core には軽量の Kestrel Web サーバーが組み込まれていますが、その前段に Apache のリバースプロキシを立てる形になります。

Microsoft Learn のページを参考にして設定を行います。

Apache 搭載の Linux で ASP.NET Core をホストする

<https://learn.microsoft.com/ja-jp/aspnet/core/host-and-deploy/linux-apache?view=aspnetcore-8.0>

しかしながら、2024 年 6 月現在では上記ページの記述は、Apache ではなく Nginx の記述に変わってしまいました。

Nginx 搭載の Linux で ASP.NET Core をホストする

<https://learn.microsoft.com/ja-jp/aspnet/core/host-and-deploy/linux-nginx?view=aspnetcore-8.0>

仕方がないので記憶している範囲で手順を記載します。Apache の mod_headers モジュールを有効にします。

```
# a2enmod headers
# systemctl restart apache2
```

Apache の設定ファイルにリバースプロキシの設定を行います。

/etc/apache2/sites-enabled/meguri.conf ファイルのリバースプロキシの設定の部分を抜粋します。4〜7 行目、23〜26 行目がリバースプロキシの設定です。5000 番ポートで起動する Kestrel に接続を中継します。Kestrel のポートは 5000 番以外にも変更可能です。

```
1 <VirtualHost *:80>
2     ServerName meguri.info
3
4     RequestHeader set "X-Forwarded-Proto" expr=%{REQUEST_SCHEME}s
5     ProxyPreserveHost On
6     ProxyPass / http://127.0.0.1:5000/
7     ProxyPassReverse / http://127.0.0.1:5000/
      (中略)
11 </VirtualHost>
12
13 <VirtualHost *:443>
14     ServerName meguri.info
      (中略)
23     RequestHeader set "X-Forwarded-Proto" expr=%{REQUEST_SCHEME}s
24     ProxyPreserveHost On
25     ProxyPass / http://127.0.0.1:5000/
26     ProxyPassReverse / http://127.0.0.1:5000/
      (中略)
30 </VirtualHost>
```

公開用のディレクトリにアプリケーションを発行します。

```
$ dotnet publish --configuration Release
$ sudo rm -rf /var/www/meguri
$ sudo cp -pr /home/meguri/repos/meguri/meguri/bin/Release/net8.0/publish /var/www/meguri
$ sudo chown www-data:www-data /var/www/meguri
```

kestrel をサービスとして起動させるため、サービスファイルを作成します。

```
# vim /etc/systemd/system/kestrel-meguri.service

[Unit]
Description=.NET Web App running on Debian

[Service]
WorkingDirectory=/var/www/meguri
ExecStart=/usr/bin/dotnet /var/www/meguri/meguri.dll
Restart=always
# Restart service after 10 seconds if the dotnet service crashes:
RestartSec=10
KillSignal=SIGINT
SyslogIdentifier=dotnet-meguri
User=www-data
Environment=ASPNETCORE_ENVIRONMENT=Production

[Install]
WantedBy=multi-user.target
```

サービスを有効にします。

```
# systemctl enable kestrel-meguri.service
```

サービスを起動し、動作を確認します。

```
# systemctl start kestrel-meguri.service
# systemctl status kestrel-meguri.service
```

第3章 PostgreSQL のセットアップ

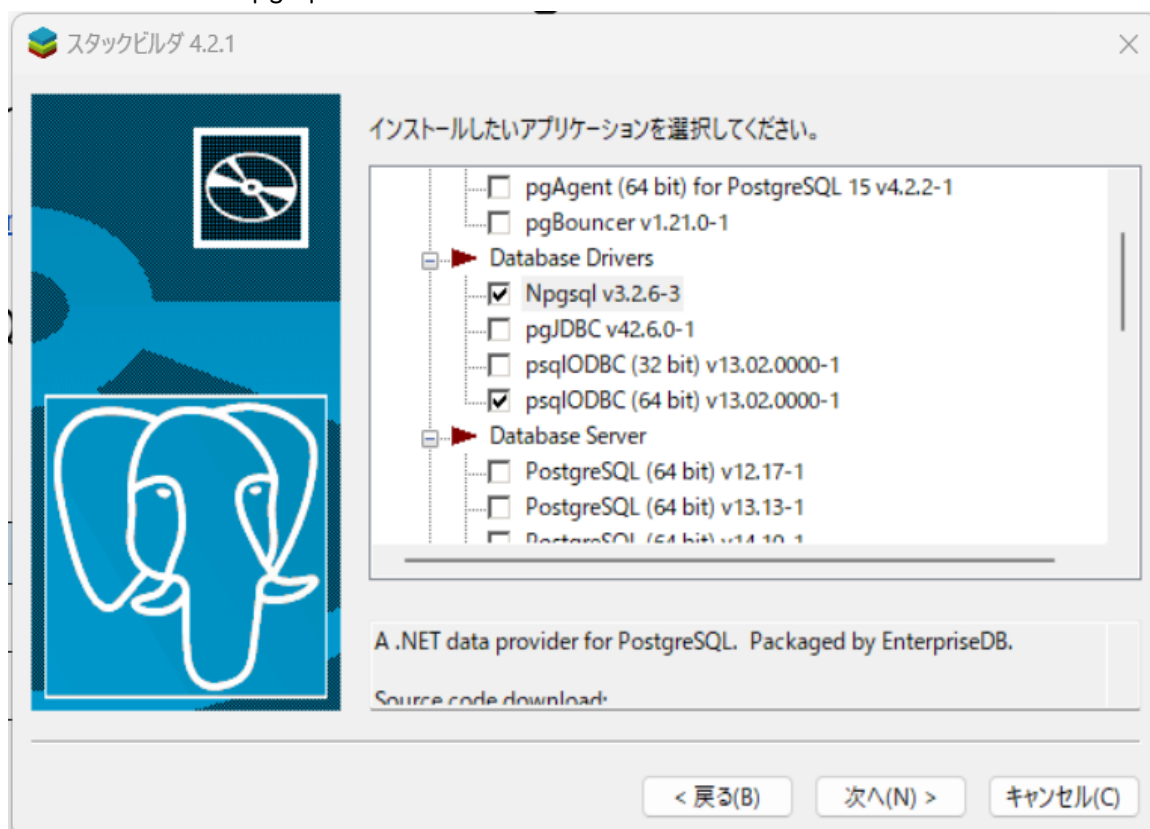
データベースには PostgreSQL を使用します。SQL Server が楽ではありますが、お高いので。PostgreSQL の .NET 用のドライバである Npgsql を使用すれば、.NET の Entity Framework のマイグレーションも問題なく動作します。

開発環境 Windows

開発環境の Windows に PostgreSQL をインストールします。下記の URL からインストーラー、postgresql-15.5-1-windows-x64.exe をダウンロードして実行します。

<https://sbp.enterprisedb.com/getfile.jsp?fileid=1258790>

インストール時に、Npgsql にチェックを入れます。



運用環境 Debian 12

運用環境の Debian 12 に PostgreSQL をインストールします。PostgreSQL のインストールについては下記ページを参考にしてください。

Debian 12 Bookworm : PostgreSQL 15 : インストール : Server World
https://www.server-world.info/query?os=Debian_12&p=postgresql&f=1

開発環境から運用環境の PostgreSQL へ接続するための設定については下記ページを参考にしてください。

Debian 12 Bookworm : PostgreSQL 15 : リモート接続の設定 : Server World
https://www.server-world.info/query?os=Debian_12&p=postgresql&f=3

PostgreSQL にリモートホストから安全に接続するには
<https://qiita.com/tom-sato/items/d5f722fd02ed76db5440>

本書では一部の手順について記載します。

pg_hba.conf の末尾に開発環境の IP アドレスを追記します。

```
# cd /etc/postgresql/15/main
# vim pg_hba.conf

# 開発環境の IP アドレス
host      all             all             192.0.2.0/24      scram-sha-256
```

パスワードの暗号化方式を設定します。

```
# vim postgresql.conf

password_encryption = scram-sha-256
```

pg_ctl コマンドの reload モードで設定ファイルの再読み込みを行い、password_encryption パラメータの設定を反映します。現在の設定は postgres コマンドの -C オプションで確認できます。

```
$ pg_ctl reload
サーバにシグナルを送信しました
$ postgres -C password_encryption
scram-sha-256
```

第4章 Postfix / Dovecot のセットアップ

.NET のチュートリアルでは、メール送信には外部サービスの SendGrid などの API の使用を推奨していますが、本書では Postfix / Dovecot を使用します。API を利用した方が楽なんですけど、できるだけ外部のサービスに依存したくなかったのです。

Postfix インストール

Postfix のインストールは下記ページを参考にしてください。

Debian 12 Bookworm : Postfix : インストール : Server World
https://www.server-world.info/query?os=Debian_12&p=mail&f=1

Dovecot インストール

Dovecot のインストールは下記ページを参考にしてください。

Debian 12 Bookworm : Dovecot : インストール : Server World
https://www.server-world.info/query?os=Debian_12&p=mail&f=2

メールアカウント登録

メールアカウントとして仮想ユーザーを登録します。下記ページを参考にしてください。

Debian 12 Bookworm : メールアカウント登録 (仮想ユーザー) : Server World
https://www.server-world.info/query?os=Debian_12&p=mail&f=9

本書では仮想ユーザーを使用していますが、メールアカウントとして OS ユーザーを使用する場合は、下記ページを参考にしてください。

Debian 12 Bookworm : メールユーザーアカウント登録 : Server World
https://www.server-world.info/query?os=Debian_12&p=mail&f=3

SSL/TLS の設定

SSL 証明書を取得します。SSL 証明書は Let's Encrypt を使用します。

```
# certbot certonly --standalone -d mail.meguri.info
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Requesting a certificate for mail.meguri.info

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/mail.meguri.info/fullchain.pem
Key is saved at: /etc/letsencrypt/live/mail.meguri.info/privkey.pem
This certificate expires on 2024-05-23.
These files will be updated when the certificate renews.

NEXT STEPS:
- The certificate will need to be renewed before it expires. Certbot can automatical
  ly renew the certificate in the background, but you may need to take steps to enable
  that functionality. See https://certbot.org/renewal-setup for instructions.

-----
If you like Certbot, please consider supporting our work by:
* Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
* Donating to EFF: https://eff.org/donate-le
-----
```

Postfix と Dovecot に SSL の設定を行います。下記ページを参考にしてください。

Debian 12 Bookworm : SSL/TLS の設定 : Server World

https://www.server-world.info/query?os=Debian_12&p=mail&f=5

SPF の設定

Amazon Route 53 に SPF レコードを設定します。下記は meguri.info での設定です。

レコード名: meguri.info

レコードタイプ: TXT

値: "v=spf1 ip4:52.197.30.185 -all"

SPF レコードの設定について詳しくは下記ページを参考にしてください。

SPF レコードを Amazon Route 53 に登録する

<https://blog.apar.jp/linux/737/>

DKIM の設定

DKIM の設定を行います。DKIM は、ドメインから送信するメールの電子署名の検証を可能にする仕組みです。DKIM では、DNS に電子署名の検証に使用する公開鍵を登録して公開します。DKIM の設定については下記のページを参考にしてください。

Debian の Postfix での SPF / DKIM / DMARC 設定 with Route 53

<https://kmuto.hatenablog.com/entry/2022/09/19/105809>

#8 Ubuntu20 にセキュアな Postfix メールサーバを構築 - 3 回目

https://www.hs3.org/Ubuntu20_Postfix_03

Postfix に DKIM を設定してみた

<https://zenn.dev/uniformnext/articles/09717539cc0b89>

本書では一部の手順について記載します。

Postfix に OpenDKIM をインストールします。

```
# apt install opendkim opendkim-tools
# gpasswd -a postfix opendkim
```

/etc/opendkim.conf を編集します。14～15 行目を編集します。54～64 行目を追記します。

```
10 # Common signing and verification parameters. In Debian, the "From" header is
11 # oversigned, because it is often the identity key used by reputation systems
12 # and thus somewhat security sensitive.
13 Canonicalization    relaxed/simple
14 Mode                sv
15 SubDomains          no

54 # Map domains in From addresses to keys used to sign messages
55 KeyTable             refile:/etc/opendkim/key.table
56 SigningTable        refile:/etc/opendkim/signing.table
57
58 # Hosts to ignore when verifying signatures
59 ExternalIgnoreList  /etc/opendkim/trusted.hosts
60
61 # A set of internal hosts whose mail should be signed
62 InternalHosts       /etc/opendkim/trusted.hosts
```

各設定ファイル用のディレクトリを作成します。

```
# mkdir /etc/opendkim
# mkdir /etc/opendkim/keys
# mkdir /etc/opendkim/keys/meguri.info
# chown -R opendkim:opendkim /etc/opendkim
# chmod 700 /etc/opendkim/keys
```

Signing Table を作成します。

```
# vim /etc/openssh/ssh-keygen.conf
#@meguri.info      default._domainkey.meguri.info
```

Key Table を作成します。

```
# vim /etc/openssh/key.table
default._domainkey.meguri.info      meguri.info:default:/etc/openssh/keys/meguri.info/default.private
```

Trusted Hosts を作成します。

```
# vim /etc/openssh/trusted.hosts
127.0.0.1
::1
localhost
meguri.info
mail.meguri.info
```

秘密鍵と公開鍵を作成します。

```
# openssl genrsa -b 2048 -d meguri.info -D /etc/openssh/keys/meguri.info -s default -v
# chown openssh:openssh /etc/openssh/keys/meguri.info/default.*
```

公開鍵を Route 53 に登録します。

Record name	Info	Record type	Info
default._domainkey .meguri.info		TXT - Used to verify email senders and for applicatio...	
Keep blank to create a record for the root domain.			
☒ Alias			
Value	Info		
<pre>"v=DKIM1; h=sha256; k=rsa; p=MIIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA7HG26yF9zVt6AhIE2QOwJvxVFhNmD0gRYrRE4Du2aTAqWW+s 4ekqcbXbMVof01n+j5RjVuCqu/MThHDuFWpYA/bdDvaf7fu+Y9k076c4dnRMbAuyHSF6NOuKisZZt/j5tyU5rV5x" "jNAPwYB4iFIMDKWFTZX0Wq2dCeSQmix1ccT0cb9j8JfGEgWCKTU4c0HiGTxLstoaXbvjr3dZg7i4/AdERPwvGfA+H5hotFZaYm LLqBew6opv2l5tAZlBx9XmRHuAidxyDBKRsZnXrs+VSoWjiNa4GXHv5B2QpNU47CwkvDG1SIOYr1sqwseH7hrSYQBeGd42YoJ L62egnWIDAQAB"</pre>			

Postfix と OpenDKIM の通信に使用するソケットのディレクトリを作成します。

```
# mkdir /var/spool/postfix/openssl
# chown openssl:postfix /var/spool/postfix/openssl
```

OpenDKIM の設定ファイル (/etc/default/openssl) を編集して、ソケットのパスを変更します。設定ファイルのコメントに legacy と書いてあるので、もしかしたら次の設定ファイル (/etc/openssl.conf) だけでいいのかも？

```
# vim /etc/default/openssl

12 # Change to /var/spool/postfix/run/openssl to use a Unix socket with
13 # postfix in a chroot:
14 RUNDIR=/var/spool/postfix/run/openssl
```

OpenDKIM の設定ファイル (/etc/openssl.conf) を編集して、ソケットのパスを変更します。

```
# vim /etc/openssl.conf

33 # Socket for the MTA connection (required). If the MTA is inside a chroot jail,
34 # it must be ensured that the socket is accessible. In Debian, Postfix runs in
35 # a chroot in /var/spool/postfix, therefore a Unix socket would have to be
36 # configured as shown on the last line below.
37 #Socket      local:/run/openssl/openssl.sock
38 #Socket      inet:8891@localhost
39 #Socket      inet:8891
40 Socket      local:/var/spool/postfix/openssl/openssl.sock
```

Postfix の設定ファイル (/etc/postfix/main.cf) の末尾にメールフィルターとして OpenDKIM を使用するように設定します。

```
# vim /etc/postfix/main.cf

# メールフィルタ設定
milter_default_action = accept
milter_protocol = 6
smtpd_milters = local:openssl/openssl.sock
non_smtpd_milters = $smtpd_milters
```

Postfix と OpenDKIM を再起動します。

```
# systemctl restart postfix openssl
```

DMARC の設定

DMARC(Domain-based Message Authentication, Reporting, and Conformance)の設定を行います。DMARCでは、SPFやDKIMで認証に失敗したメールの処理方法、認証に失敗したメールについてのレポートの宛先などの情報をDNSに登録して公開します。下記のページを参考にしてください。

DebianのPostfixでのSPF / DKIM / DMARC設定 with Route 53

<https://kmuto.hatenablog.com/entry/2022/09/19/105809>

#8 Ubuntu20 にセキュアなPostfixメールサーバを構築 - 3 回目

<https://www.hs3.org/Ubuntu20 Postfix 03>

DMARCで詐欺メールを排除!簡単なDMARCの設定方法 openDMARC

<https://www.tyksnet.com/blog/archives/2593>

本書では一部の手順について記載します。

Route 53にDMARCレコードに登録します。

Record name	Info	Record type	Info
_dmarc	.meguri.info	TXT - Used to verify email senders and for applicatio...	
Keep blank to create a record for the root domain.			
☒ Alias			
Value	Info		
<input type="text" value='"v=DMARC1; p=reject; rua=mailto:meguri.mail@gmail.com, mailto:meguri.mail@gmail.com; pct=100; adkim=s; aspf=s"'/>			

OpenDMARCの設定ファイルを編集します。変更箇所のみ抜粋します。

```
# vim /etc/opendmarc.conf

13 # AuthservID name
14 AuthservID OpenDMARC

53 # RejectFailures false
54 RejectFailures true

69 #Socket local:/run/opendmarc/opendmarc.sock
70 Socket local:/var/spool/postfix/opendmarc/opendmarc.sock

96 # TrustedAuthservIDs HOSTNAME
97 TrustedAuthservIDs mail.meguri.info

119 UserID opendmarc
120
121 IgnoreHosts /etc/opendmarc/ignore.hosts
```

```
122 SPFIgnoreResults true
123 SPFSelfValidate true
124 RejectFailures true
125 RequiredHeaders true
126 RequiredHeaders true
127 IgnoreAuthenticatedClients true
```

Postfix と OpenDMARC の通信に使用するソケットのディレクトリを作成します。

```
# mkdir -p /var/spool/postfix/openssl
# chown -R openssl:openssl /var/spool/postfix/openssl
# chmod 750 -R /var/spool/postfix/openssl
# gpasswd -a postfix openssl
```

Postfix の設定ファイルを編集します。smtpd_milters に、local:openssl/openssl.sock を追加します。

```
# vim /etc/postfix/main.cf

smtpd_milters = local:openssl/openssl.sock
```

Postfix と OpenDMARC を再起動します。

```
# systemctl restart postfix openssl
```

第 5 章 Web アプリケーション開発

.NET 8 SDK のインストール

下記ページを参考に Debian 12 に .NET 8 SDK をインストールします。

Debian に .NET SDK または .NET ランタイムをインストールする

<https://learn.microsoft.com/ja-jp/dotnet/core/install/linux-debian>

信頼されたキーの一覧に Microsoft パッケージ署名キーを追加し、パッケージリポジトリを追加します。

```
# wget https://packages.microsoft.com/config/debian/12/packages-microsoft-prod.deb -  
O packages-microsoft-prod.deb  
# dpkg -i packages-microsoft-prod.deb  
# rm packages-microsoft-prod.deb
```

.NET SDK をインストールします。

```
# apt-get update && ¥  
apt-get install -y dotnet-sdk-8.0
```

プロジェクトの作成

下記のページを参考に Debian 12 上で ASP.NET Core MVC アプリケーションのプロジェクトを作成します。下記のページを参考にします。

パート 2.1 - Linux での ASP.NET Core アプリケーションの作成と構成

<https://learn.microsoft.com/ja-jp/troubleshoot/developer/webapps/aspnetcore/practice-troubleshoot-linux/2-1-create-configure-aspnet-core-applications?source=recommendations>

C++(Visual Studio)で UTF-8 を扱うための試行錯誤のメモ

<https://qiita.com/BlueSilverCat/items/1e5af7fdaeaf8fd2289d>

Linux で CRLF と LF を一括で相互変換する

https://zenn.dev/hidemaru_owo/articles/d6701c1f3bb0e1

.NET CLI の概要

<https://learn.microsoft.com/ja-jp/dotnet/core/tools/>

ソリューション及びプロジェクトを作成します。--auth Individual オプションを指定して、「認証あり」のプロジェクトを作成します。

```
$ cd
$ mkdir repos
$ cd repos
$ dotnet new sln --output meguri
$ cd meguri
$ dotnet new mvc --auth Individual --use-local-db --use-program-main --output meguri
$ dotnet sln add meguri
```

プロジェクトを Debian 上で作成しても文字コードと改行コードが UTF-8 と LF に統一されなかったの
で nkf でカレントディレクトリ内のファイルを全て変換変換します。

```
$ grep -lrs `printf "%r\n"` . | xargs nkf -Lu --overwrite
```

Meguri のソースコード全体は GitHub の下記のリポジトリを参照してください。

<https://github.com/megurikuru/meguri>

コードファーストマイグレーションの実行

PostgreSQL でも Npgsql をインストールすればコードファーストマイグレーションの実行が可能です。
詳しくは下記ページを参考にしてください。

【EF Core】コードファーストで PostgreSQL のデータベースを作成する【ASP.NET】

<https://qiita.com/1noseA/items/861b64549fe7083e4347>

Program.cs を修正します。データベースに Microsoft SQL Server ではなく PostgreSQL を使用
する場合は AddDbContext のオプションを UseSqlServer から UseNpgsql に変更します。

```
builder.Services.AddDbContext<ApplicationDbContext>(
    options => options.UseNpgsql(connectionString)
);
```

マイグレーションファイルを作成してデータベースを作成します。

```
Add-Migration InitialCreate
Update-Database
```

認証

よく認証認可とまとめて言いますが、認証は ID とパスワード等でユーザーを一意に識別することで、認
可は一意に識別したユーザーに対して、システムにおける操作、例えば、自身が作成したコンテンツの編
集及び削除を許可することです。本節では認証について説明し、次節で認可について説明します。

.NET Core MVC または Razor Pages で「認証あり」でプロジェクトを作成していれば基本的な認証機能は最初から組み込まれています。ただ、メール送信と多要素認証の QR コードの生成については手を入れる必要があるので、この後説明します。

.NET Core の認証について詳しくは下記のページを参照してください。

ASP.NET Core の認証の概要

<https://learn.microsoft.com/ja-jp/aspnet/core/security/authentication/?view=aspnetcore-8.0>

メール送信

メール送信機能を追加します。メール送信には System.Net.Mail.SmtpClient ではなく、サードパーティーの MailKit を使用します。現在、SmtpClient の使用は推奨されていません。NuGet パッケージマネージャーで MailKit をプロジェクトに追加します。

下記のページを参考にしてください。

ASP.NET Core でのアカウントの確認とパスワードの回復

<https://learn.microsoft.com/ja-jp/aspnet/core/security/authentication/acconfirm?view=aspnetcore-8.0&tabs=visual-studio>

【C#】メールを送信する方法を紹介 (MailKit)

<https://marunaka-blog.com/csharp-mail-send-mailkit/10081/>

[SMTP]MailKit を用いた認証方式しらべ

<https://zenn.dev/nasrisilva/articles/ae31f5597d653e>

電子メールを送信するには? (MailKit 編) [.NET 4.5, C# / VB]

<https://atmarkit.itmedia.co.jp/ait/articles/1811/21/news023.html>

MailKit/README.md at master · jstedfast/MailKit

<https://github.com/jstedfast/MailKit/blob/master/README.md>

SmtpClient.AuthenticationMechanisms Property

https://mimekit.net/docs/html/P_MailKit_Net_Smtp_SmtpClient_AuthenticationMechanisms.htm

C#でSMTP (STARTTLS と SMTP over SSL) でメール送信する

<https://yoshinorin.net/articles/2016/10/29/csharp-mail-smtps-starttls/>

メール送信のために追加や編集したファイルについて説明します。

ISender インターフェイスを実装した EmailSender クラスを作成します。EmailSender クラスは、Services ディレクトリを作成して、その配下に保存します。

```

using MailKit.Net.Smtp;
using MailKit.Security;
using Microsoft.AspNetCore.Identity.UI.Services;
using Microsoft.Extensions.Options;
using MimeKit;

namespace Meguri.Services;

public class EmailSender : IEmailSender {
    private readonly ILogger _logger;

    public EmailSender(
        IOptions<SMTPServerConf> optionsAccessor, ILogger<EmailSender> logger
    ) {
        Options = optionsAccessor.Value;
        _logger = logger;
    }

    public SMTPServerConf Options { get; }

    public async Task SendEmailAsync(
        string toEmail, string subject, string message
    ) {
        await Execute(subject, message, toEmail);
    }

    public async Task Execute(string subject, string message, string toEmail) {
        var mimeMessage = new MimeMessage();
        mimeMessage.From.Add(new MailboxAddress(
            Options.Account, Options.Account)
        );
        mimeMessage.To.Add(new MailboxAddress(toEmail, toEmail));
        mimeMessage.Subject = subject;
        mimeMessage.Body = new TextPart("html") {
            Text = message
        };
        try {
            using (var client = new SmtplibClient()) {
                client.LocalDomain = Options.LocalDomain;
                await client.ConnectAsync(
                    Options.HostName, Options.Port, SecureSocketOptions.Auto
                );
                await client.AuthenticateAsync(
                    Options.Account, Options.Password
                );
                await client.SendAsync(mimeMessage);
                await client.DisconnectAsync(true);
            }
        }
    }
}

```

```

    } catch (Exception ex) {
        _logger.LogInformation(
            "Failure Email to {ToEmail}. {ExMessage}", toEmail, ex.Message
        );
        return;
    }
    _logger.LogInformation(
        "Email to {ToEmail} queued successfully!", toEmail
    );
}
}

```

appsettings.json に SMTP サーバーの設定項目を追加します。

appsettings.json

```

"SMTPServerConf": {
  "HostName": "mail.example.com",
  "Port": 465,
  "LocalDomain": "example.com",
  "Accout": "account@example.com",
  "Password": "PASSWORD"
}

```

SMTP サーバーの設定を読み込むクラスを作成します。Services ディレクトリの下に保存します。

Services/SMTPServerConf.cs

```

namespace Meguri.Services;

public class SMTPServerConf {
    public string? HostName { get; set; }
    public int Port { get; set; }
    public string? LocalDomain { get; set; }
    public string? Accout { get; set; }
    public string? Password { get; set; }
}

```

Progrma.cs に SMTP サーバーの設定の読み込み、メール送信サービスの追加、サービスへの設定の登録を行うコードを追加します。

Progrma.cs

```

var smtpServerConf = builder.Configuration.GetSection("SMTPServerConf");
builder.Services.AddTransient<IEmailSender, EmailSender>();
builder.Services.Configure<SMTPServerConf>(smtpServerConf);

```


多要素認証のQRコード生成

「認証あり」でプロジェクトを作成していれば、多要素認証の機能もデフォルトで組み込まれています。コードを書く必要はありません。詳しくは下記ページを参照してください。

ASP.NET Core での多要素認証

<https://learn.microsoft.com/ja-jp/aspnet/core/security/authentication/mfa?view=aspnetcore-8.0>

ASP.NET Core アプリに 2 要素認証を実装

<http://surferonwww.info/BlogEngine/post/2021/11/06/aspnet-core-application-with-two-factor-authentication.aspx>

ただ、デフォルトでは QR コードの生成機能はないので qrcode.js を追加します。詳しくは下記ページを参考にしてください。

ASP.NET Core で TOTP 認証アプリ用の QR コードを生成できるようにする

<https://learn.microsoft.com/ja-jp/aspnet/core/security/authentication/identity-enable-qr-codes?view=aspnetcore-8.0>

認可

自分が作成したコンテンツは自分だけが編集できるようにします。下記のページを参考にします。

認可によって保護されたユーザー データを使って ASP.NET Core Web アプリを作成する

<https://learn.microsoft.com/ja-jp/aspnet/core/security/authorization/secure-data?view=aspnetcore-8.0>

ASP.NET Core MVC と EF Core - チュートリアル シリーズ

<https://learn.microsoft.com/ja-jp/aspnet/core/data/ef-mvc/?view=aspnetcore-8.0>

モデルの修正

コンテンツを格納する Work モデルに UserId を追加します。

Models/Work.cs

```
namespace Meguri.Models {
    [Table("Works")]
    [Index(nameof(ParentId))]
    [Index(nameof(CategoryId))]
    [Index(nameof(Tag1))]
    [Index(nameof(Tag2))]
```

```

[Index(nameof(Tag3))]
[Index(nameof(Created))]
[Index(nameof(Updated))]
public class Work {
    public int Id { get; set; }
    public string? UserId { get; set; }
    public string? UserName { get; set; }
    public string? Name { get; set; }
    public string? Text { get; set; }
    public int ParentId { get; set; }
    public string? WorkType { get; set; }
    public int CategoryId { get; set; }
    public string? Tag1 { get; set; }
    public string? Tag2 { get; set; }
    public string? Tag3 { get; set; }
    public byte[]? FileContent { get; set; }
    public bool Sexual { get; set; } = false;
    public bool Violence { get; set; } = false;
    public DateTime Created { get; set; }
    public DateTime Updated { get; set; }

    public Category? Category { get; set; }
}
}

```

UserId は IdentityUser を継承してつくった User クラスの主キーです。User クラスは IdentityUser にいくつかのメンバーを追加しています。マイグレーションを実行すると AspNetUsers テーブルにカラムが追加されます。

Models/User.cs

```

namespace Meguri.Models {
    [Index(nameof(Birthday))]
    public class User: IdentityUser{
        public DateOnly Birthday { get; set; }
        public bool AllowSexual { get; set; } = false;
        public bool AllowViolence { get; set; } = false;

        public ICollection<Work>? Works { get; set; }
    }
}

```

要件クラスと認可ハンドラーの作成

要件クラス WorkOperations を作成します。WorkOperations は、Authorization ディレクトリを作成して、その配下に保存します。

Authorization/WorkOperations.cs

```
using Microsoft.AspNetCore.Authorization.Infrastructure;

namespace Meguri.Authorization {
    public static class WorkOperations {
        public static OperationAuthorizationRequirement Create = new
            OperationAuthorizationRequirement { Name = Constants.CreateOperationName };

        public static OperationAuthorizationRequirement Read = new
            OperationAuthorizationRequirement { Name = Constants.ReadOperationName };

        public static OperationAuthorizationRequirement Update = new
            OperationAuthorizationRequirement { Name = Constants.UpdateOperationName };

        public static OperationAuthorizationRequirement Delete = new
            OperationAuthorizationRequirement { Name = Constants.DeleteOperationName };

        public static OperationAuthorizationRequirement Approve = new
            OperationAuthorizationRequirement { Name = Constants.ApproveOperationName };

        public static OperationAuthorizationRequirement Reject = new
            OperationAuthorizationRequirement { Name = Constants.RejectOperationName };
    }

    public class Constants {
        public static readonly string CreateOperationName = "Create";
        public static readonly string ReadOperationName = "Read";
        public static readonly string UpdateOperationName = "Update";
        public static readonly string DeleteOperationName = "Delete";
        public static readonly string ApproveOperationName = "Approve";
        public static readonly string RejectOperationName = "Reject";

        public static readonly string ContactAdministratorsRole =
            "ContactAdministrators";
        public static readonly string ContactManagersRole = "ContactManagers";
    }
}
```

コンテンツの所有者を確認する認可ハンドラー `WorkIsOwnerAuthorizationHandler` を作成します。
`WorkIsOwnerAuthorizationHandler` は、`Authorization` ディレクトリの配下に保存します。

Authorization/WorkIsOwnerAuthorizationHandler.cs

```
using Meguri.Models;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Authorization.Infrastructure;
using Microsoft.AspNetCore.Identity;
```

```

namespace Meguri.Authorization {
    public class WorkIsOwnerAuthorizationHandler
        : AuthorizationHandler<OperationAuthorizationRequirement, Work> {
        UserManager<IdentityUser> _userManager;

        public WorkIsOwnerAuthorizationHandler(
            UserManager<IdentityUser> userManager
        ) {
            _userManager = userManager;
        }

        protected override Task HandleRequirementAsync(
            AuthorizationHandlerContext context,
            OperationAuthorizationRequirement requirement,
            Work resource
        ) {
            if (context.User == null || resource == null) {
                return Task.CompletedTask;
            }

            // If not asking for CRUD permission, return.
            if (requirement.Name != Constants.CreateOperationName &&
                requirement.Name != Constants.ReadOperationName &&
                requirement.Name != Constants.UpdateOperationName &&
                requirement.Name != Constants.DeleteOperationName) {
                return Task.CompletedTask;
            }

            if (resource.UserId == _userManager.GetUserId(context.User)) {
                context.Succeed(requirement);
            }

            return Task.CompletedTask;
        }
    }
}

```

サービスコレクションへの登録

作成した認可ハンドラーをサービスコレクションに登録します。Program.cs の WebApplicationBuilder の Build メソッドの呼び出し前に追加します。

Program.cs

```

// 認可ハンドラー
builder.Services.AddScoped<
    IAuthorizationHandler, WorkIsOwnerAuthorizationHandler
>();

```

```
var app = builder.Build();
```

コントローラーの修正

WorksController の POST の Create アクションメソッドにコンテンツの所有者かどうかを確認するコードを追加します。

Controllers/WorksController.cs

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(
    int id, string name, string? text, int parentId, string workType,
    int categoryId, string? tag1, string? tag2, string? tag3,
    IFormFile? fileContent, bool sexual, bool violence,
    DateTime created, DateTime updated
) {
    Work work = new Work();
    if (ModelState.IsValid) {
        work.Id = id;
        work.Name = name;
        work.Text = text;
        work.ParentId = parentId;
        work.WorkType = workType;
        work.CategoryId = categoryId;
        work.Tag1 = tag1;
        work.Tag2 = tag2;
        work.Tag3 = tag3;
        work.FileContent = null;
        if (fileContent != null) {
            byte[] byteArray;
            using (var memoryStream = new MemoryStream()) {
                fileContent.CopyTo(memoryStream);
                byteArray = memoryStream.ToArray();
            }
            work.FileContent = byteArray;
        }
        work.Sexual = sexual;
        work.Created = DateTime.Now;
        work.Updated = DateTime.Now;

        var userId = UserManager.GetUserId(User);
        var userName = UserManager.GetUserName(User);
        work.UserId = userId;
        work.UserName = userName;

        // 認可
```

```

        var isAuthorized = await AuthorizationService.AuthorizeAsync(
            User, work, WorkOperations.Create
        );
        if (!isAuthorized.Succeeded) {
            return Forbid();
        }

        _context.Add(work);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(work);
}

```

WorksController の GET と POST の Edit アクションメソッドにコンテンツの所有者かどうかを認可ハンドラーで確認するコードを追加します。

Controllers/WorksController.cs

```

public async Task<IActionResult> Edit(int? id) {
    if (id == null) {
        return NotFound();
    }

    var work = await _context.Work.FindAsync(id);
    if (work == null) {
        return NotFound();
    }

    // 認可
    var isAuthorized = await AuthorizationService.AuthorizeAsync(
        User, work, WorkOperations.Update
    );
    if (!isAuthorized.Succeeded) {
        return Forbid();
    }

    return View(work);
}

```

Controllers/WorksController.cs

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(
    int id, string name, string? text, int parentId, string workType,
    int categoryId, string? tag1, string? tag2, string? tag3,
    IFormFile? fileContent, bool sexual, bool violence, DateTime created, DateTime u

```

```

pdated,
    string userId
) {
    Work? work = _context.Work.Find(id);
    if (work == null) {
        return NotFound();
    }

    // 認可
    var isAuthorized = await AuthorizationService.AuthorizeAsync(
        User, work, WorkOperations.Update
    );
    if (!isAuthorized.Succeeded) {
        return Forbid();
    }

    work.Id = id;
    work.Name = name;
    work.Text = text;
    work.ParentId = parentId;
    work.WorkType = workType;
    work.CategoryId = categoryId;
    work.Tag1 = tag1;
    work.Tag2 = tag2;
    work.Tag3 = tag3;
    if (fileContent != null) {
        byte[] UpdateFileContent;
        using (var memoryStream = new MemoryStream()) {
            fileContent.CopyTo(memoryStream);
            UpdateFileContent = memoryStream.ToArray();
        }
        work.FileContent = UpdateFileContent;
    }
    work.Sexual = sexual;
    work.Created = created;
    work.Updated = DateTime.Now;
    work.UserId = userId;

    try {
        _context.Update(work);
        await _context.SaveChangesAsync();
    } catch (DbUpdateConcurrencyException) {
        if (!WorkExists(work.Id)) {
            return NotFound();
        } else {
            throw;
        }
    }
}

```

```
    return RedirectToAction(nameof(Index));  
}
```

WorksController の Delete と DeleteConfirmed アクションメソッドに、認可ハンドラーでコンテンツの所有者かどうかを確認するコードを追加します。

Controllers/WorksController.cs

```
public async Task<IActionResult> Delete(int? id) {  
    if (id == null) {  
        return NotFound();  
    }  
  
    var work = await _context.Work  
        .FirstOrDefaultAsync(m => m.Id == id);  
    if (work == null) {  
        return NotFound();  
    }  
  
    // 認可  
    var isAuthorized = await AuthorizationService.AuthorizeAsync(  
        User, work, WorkOperations.Delete  
    );  
    if (!isAuthorized.Succeeded) {  
        return Forbid();  
    }  
  
    return View(work);  
}
```

Controllers/WorksController.cs

```
[HttpPost, ActionName("Delete")]  
[ValidateAntiForgeryToken]  
public async Task<IActionResult> DeleteConfirmed(int id) {  
    var work = await _context.Work.FindAsync(id);  
  
    // 認可  
    var isAuthorized = await AuthorizationService.AuthorizeAsync(  
        User, work, WorkOperations.Delete  
    );  
    if (!isAuthorized.Succeeded) {  
        return Forbid();  
    }  
  
    if (work != null) {  
        _context.Work.Remove(work);  
    }  
}
```



```
await _context.SaveChangesAsync();
return RedirectToAction(nameof(Index));
}
```

認可に必要なコードの追加及び修正が完了したら、複数のユーザーでコンテンツを作成し、別のユーザーが作成したコンテンツは編集及び削除ができないこと、自分が作成したコンテンツは編集及び削除ができることを確認します。

画像のアップロードとデータベースへの格納

画像については、データベースではなく通常のファイルシステムに保存したり、Amazon S3 などのストレージサービスに保存する実装も考えられますが、本書ではデータベースに保存します。画像をデータベースに保存しない場合、データベースの肥大化だけは避けられますが、よく考えてみたらそれ以外のメリットはあまりない気もするので。認証も困難になりますし。商用サービスでも画像を Amazon S3 に保存してて認証がかかってない怖いサービスを見かけます。

以下のページを参考にします。

ASP.NET Core でファイルをアップロードする

<https://learn.microsoft.com/ja-jp/aspnet/core/mvc/models/file-uploads?view=aspnetcore-8.0>

MVC でファイルのアップロード

<http://surferonwww.info/BlogEngine/post/2019/08/03/file-upload-on-aspnet-mvc-application.aspx>

[ASP.NET MVC] ASP.NET MVC でアップロードしたファイルをデータベースに登録するには? [3.5、C#、VB]

<https://atmarkit.itmedia.co.jp/fdotnet/dotnettips/939aspmvcfileupload2/aspmvcfileupload2.html>

input type file で取得したファイルを C# の ASP.NET mvc で db に保存する方法を教えてください

https://detail.chiebukuro.yahoo.co.jp/qa/question_detail/q14280125679

パート 6: ASP.NET Core のコントローラーのメソッドとビュー

<https://learn.microsoft.com/ja-jp/aspnet/core/tutorials/first-mvc-app/controller-methods-views?view=aspnetcore-8.0>

モデルの修正

バイナリデータを格納するプロパティの型を Byte 配列に指定します。

```
public byte[]? FileContent { get; set; }
```

ビューの修正

下記のファイルを修正します。

- Index.cshtml
- Details.cshtml
- Create.cshtml
- Edit.cshtml

画像の表示は、Byte 配列を Base64 エンコードして HTML に直接埋め込んでいます。とりあえず、画像ファイルのサイズがあまり小さくなく、PNG であることが前提です。

Views/Works/Index.cshtml

```
<td>
    @{
        if (item.FileContent != null) {
            var image = Convert.ToBase64String(item.FileContent);
            
        }
    }
</td>
```

Views/Works/Details.cshtml

```
<td>
    @{
        if (item.FileContent != null) {
            var image = Convert.ToBase64String(item.FileContent);
            
        }
    }
</td>
```

form に `enctype="multipart/form-data"` を追加しています。input の type を file にします。

Views/Works/Create.cshtml

```
<form asp-action="Create" enctype="multipart/form-data">
    (省略)
    <div class="form-group">
        <label asp-for="FileContent" class="control-label"></label>
        <input asp-for="FileContent" class="form-control" type="file" />
    </div>
```

```
(省略)
</form>
```

Views/Works/Edit.cshtml

```
<form asp-action="Edit" enctype="multipart/form-data">
  (省略)
  @if (Model.FileContent != null) {
    <div>
      @{
        var image = Convert.ToBase64String(Model.FileContent);
      }
      
    </div>
  }
  <input asp-for="FileContent" class="form-control" type="file" />
  <span asp-validation-for="FileContent" class="text-danger"></span>
  (省略)
</form>
```

コントローラーの修正

コントローラーの POST の Create アクションメソッドを修正します。引数で `IFormFile` を受け取るようにします。`IFormFile` をメモリストリームを使用してバイト配列に格納します。

Controllers/WorksController.cs

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(
    int id, string name, string? text, int parentId, string workType,
    int categoryId, string? tag1, string? tag2, string? tag3,
    IFormFile? fileContent, bool sexual, bool violence,
    DateTime created, DateTime updated
) {
    Work work = new Work();
    if (ModelState.IsValid) {
        work.Id = id;
        work.Name = name;
        work.Text = text;
        work.ParentId = parentId;
        work.WorkType = workType;
        work.CategoryId = categoryId;
        work.Tag1 = tag1;
        work.Tag2 = tag2;
        work.Tag3 = tag3;
        work.FileContent = null;
    }
}
```

```

        if (fileContent != null) {
            byte[] byteArray;
            using (var memoryStream = new MemoryStream()) {
                fileContent.CopyTo(memoryStream);
                byteArray = memoryStream.ToArray();
            }
            work.FileContent = byteArray;
        }
        work.Sexual = sexual;
        work.Created = DateTime.Now;
        work.Updated = DateTime.Now;

        var userId = UserManager.GetUserId(User);
        var userName = UserManager.GetUserName(User);
        work.UserId = userId;
        work.UserName = userName;

        // 認可
        var isAuthorized = await AuthorizationService.AuthorizeAsync(
            User, work, WorkOperations.Create
        );
        if (!isAuthorized.Succeeded) {
            return Forbid();
        }

        _context.Add(work);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(work);
}

```

コントローラーの POST の Edit アクションメソッドを修正します。引数で IFormFile を受け取るようにします。IFormFile をメモリストリームを使用してバイト配列に格納します。

Controllers/WorksController.cs

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(
    int id, string name, string? text, int parentId, string workType,
    int categoryId, string? tag1, string? tag2, string? tag3,
    IFormFile? fileContent, bool sexual, bool violence,
    DateTime created, DateTime updated,
    string userId
) {
    Work? work = _context.Work.Find(id);
    if (work == null) {

```

```

        return NotFound();
    }

    // 認可
    var isAuthorized = await AuthorizationService.AuthorizeAsync(
        User, work, WorkOperations.Update
    );

    work.Id = id;
    work.Name = name;
    work.Text = text;
    work.ParentId = parentId;
    work.WorkType = workType;
    work.CategoryId = categoryId;
    work.Tag1 = tag1;
    work.Tag2 = tag2;
    work.Tag3 = tag3;
    if (fileContent != null) {
        byte[] UpdateFileContent;
        using (var memoryStream = new MemoryStream()) {
            fileContent.CopyTo(memoryStream);
            UpdateFileContent = memoryStream.ToArray();
        }
        work.FileContent = UpdateFileContent;
    }
    work.Sexual = sexual;
    work.Created = created;
    work.Updated = DateTime.Now;
    work.UserId = userId;

    try {
        _context.Update(work);
        await _context.SaveChangesAsync();
    } catch (DbUpdateConcurrencyException) {
        if (!WorkExists(work.Id)) {
            return NotFound();
        } else {
            throw;
        }
    }

    return RedirectToAction(nameof(Index));
}

```

Meguri でのコンテンツの分類

Meguri ではとりあえず下記の 14 の分類を考えています。X (旧 Twitter)、Instagram、YouTube などは、それぞれ短文投稿、写真投稿、動画投稿に特化していますが、Meguri では、コンテンツの分類によって、最適な UI を切り替えられたらいいなと思っています。

No.	日本語	英語
1	イラスト	Illustration
2	漫画	Comic
3	小説	Novel
4	ノンフィクション	Nonfiction
5	技術書	TechnicalBook
6	写真	Photo
7	動画	Movie
8	ゲーム	Game
9	プログラム	Program
10	音楽	Music
11	コスプレ	Cosplay
12	ブログ	Blog
13	投稿	Post
14	技術ブログ	TechnicalBlog

.NET on Debian で 同人活動に優しい SNS の開発 基礎編
発行日: 2024 年 8 月 12 日
発行者: meguri
連絡先: meguri.mail@gmail.com
印刷所: 株式会社栄光